# The Tennis Racket Theorem: A Computational Study of the Intermediate Axis Instability

Computational Physics (EP4210)

Aryan Raval (EP25BTECH11007), Vishal Yalavarti (EP25BTECH11033), Yug Gandhi (EP25BTECH11035)

10 November, 2026

**Abstract**

The Tennis Racket Theorem also known as the Dzhanibekov effect is when a body rotating close to its interdiate axis (explained below) periodically flips its orientation in space. Our project involved observing this effect, simulating the rotation of rigid bodies given their initial orientation and angular velocities using two models, (1) A set of masses connected by rigid springs and (2) Using Quaternions and Euler Equations to store orientation. The first model is basically a simple physics engine that was written in both python and rust and the second model written in python, is a more direct approach that updates the orientation of the body using rk4 integration. We see that both our models agree as the stiffness of the springs is increased in the first model and the timestep is reduced. An extension to this project that we explored was adding damping to the springs to observe the fact that a non-rigid body that can loose energy by 'wobbling' will eventually stabilise its rotation to the axis with maximum moment of inertia. This also explains why the rotation of Earth is stable and why it won't flip upside down.

## 1. Introduction

### *1.1. A short history of the effect*

The equations governing a rotating rigid body with no torque acting on it were written down by Euler in the 1760s, and in 1834 Poinsot gave the geometric picture of the motion that is still used today: the angular momentum vector rolls along curves on the body's energy ellipsoid [5]. From this analysis it follows that steady rotation about the axes of largest and smallest moment of inertia is stable, while rotation about the intermediate axis is not.

The most famous observation came from space. In June 1985 the cosmonauts Vladimir Dzhanibekov and Viktor Savinykh flew Soyuz T-13 to the Salyut 7 space station, which had lost power and stopped responding to ground control months earlier. They docked manually with the tumbling, frozen station and brought it back to life. While unpacking supplies, Dzhanibekov unscrewed a wing nut that spun off the end of its bolt and floated away. As it drifted across the cabin, the nut flipped by 180° every few seconds, again and again, while its spin stayed otherwise the same. In microgravity there was nothing to cut the motion short, so the flips repeated periodically, which had never been seen so clearly before. The phenomenon is now often called the *Dzhanibekov effect* [13]. Popular accounts claim the observation was kept quiet by the Soviet authorities for about a decade and was later linked to speculation that the Earth itself might flip over [10]. We now know that the Earth won't flip over because it is already spinning about its axis with the maximum moment of inertia.

A related lesson had been learned earlier with Explorer 1, the first American satellite, launched in 1958. It was designed to spin about its long axis, the axis of *minimum* moment of inertia, but its flexible antennas dissipated energy and within hours it had moved to a tumble about its axis of maximum moment. For a body that is not perfectly rigid, only the major axis is truly stable.

The effect is easy to see on Earth. A tennis racket, a book or a phone tossed in the air while spinning about its intermediate axis comes back having turned over by half a turn about another axis, while spins about the other two axes stay clean.

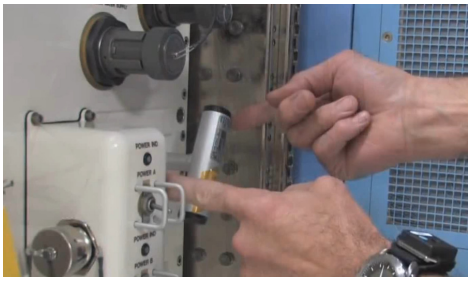


**Figure 1:** Demonstration of the effect by NASA astronauts in microgravity [13].

## 2. Theory

### 2.1. Rigid body kinematics

We describe the body with two frames: a lab frame fixed in space, and a body frame with axes $\mathbf{e}_1, \mathbf{e}_2, \mathbf{e}_3$ that rotate with the body. The rotation is described by the angular velocity $\boldsymbol{\omega}$, and each body axis moves as $\dot{\mathbf{e}}_k = \boldsymbol{\omega} \times \mathbf{e}_k$. We write $\Omega_k = \boldsymbol{\omega} \cdot \mathbf{e}_k$ for the components of $\boldsymbol{\omega}$ in the body frame. For a free body no torque acts, so the angular momentum $\mathbf{L}$ is fixed in the lab frame. Seen from the body, however, $\mathbf{L}$ moves, and following this motion is the key to the effect.

### 2.2. The inertia tensor

For a body made of point masses $m_i$ at positions $\mathbf{r}_i$ from the centre of mass, the inertia tensor is

$$I_{ab} = \sum_i m_i \left( r_i^2 \, \delta_{ab} - r_{i,a} \, r_{i,b} \right) \tag{1}$$

and $\mathbf{L} = \mathsf{I}\,\boldsymbol{\omega}$. In general $\mathbf{L}$ and $\boldsymbol{\omega}$ are not parallel. Since $\mathsf{I}$ is symmetric, there are three perpendicular *principal axes* in which it is diagonal, with principal moments $I_1 > I_2 > I_3$. Spinning about any one of them keeps $\mathbf{L}$ parallel to $\boldsymbol{\omega}$. Our main body is an octahedron of six 1 kg masses at $(\pm 1, 0, 0)$, $(0, \pm 2, 0)$ and $(0, 0, \pm 3)$ m, whose principal axes are $x$, $y$ and $z$ with $I_x = 26$, $I_y = 20$ and $I_z = 10\,\mathrm{kg\,m^2}$, so $y$ is the middle axis.

### 2.3. Euler's equations

Written in the body frame along the principal axes, $\dot{\mathbf{L}} = 0$ becomes Euler's equations,

$$\begin{aligned} I_1 \dot{\Omega}_1 &= (I_2 - I_3)\,\Omega_2 \Omega_3 \\ I_2 \dot{\Omega}_2 &= (I_3 - I_1)\,\Omega_3 \Omega_1 \\ I_3 \dot{\Omega}_3 &= (I_1 - I_2)\,\Omega_1 \Omega_2 \end{aligned} \tag{2}$$

They conserve the kinetic energy and the magnitude of the angular momentum,

$$2E = \sum_k \frac{L_k^2}{I_k}, \qquad L^2 = \sum_k L_k^2, \tag{3}$$

with $L_k = I_k \Omega_k$. These two quantities are what we monitor to check our simulations (section 7.2).

### 2.4. Instability of the middle axis

Suppose the body spins with rate $\Omega$ about one principal axis and is given a small tilt. It can be shown that about the axes of largest and smallest moment the tilt only oscillates, but about the intermediate axis it grows as $e^{\lambda t}$ with

$$\lambda = \Omega \sqrt{\frac{(I_1 - I_2)(I_2 - I_3)}{I_1 I_3}}. \tag{4}$$

For the octahedron this gives $\lambda \approx 0.480\,\Omega$. If the initial tilt has size $\varepsilon$, it becomes of order one, which is a flip, after roughly

$$t_{\mathrm{flip}} \approx \frac{1}{\lambda} \ln \frac{1}{\varepsilon}, \tag{5}$$

so the flip time grows only slowly as the tilt is made smaller. We test this in section 6.3.

### 2.5. Geometric picture: the polhode

Equation (3) gives a simple picture of the motion. In the body frame $\mathbf{L}$ must lie on a sphere of radius $L$ and, at the same time, on the energy ellipsoid. It therefore moves along the curve where the two surfaces meet, called the polhode (fig. 2). Near the largest and smallest axes these curves are small closed loops, so $\mathbf{L}$ stays close to the axis and the rotation is stable. Near the middle axis the two surfaces cross, and the special curve through this axis, the separatrix, carries $\mathbf{L}$ all the way to the opposite side.

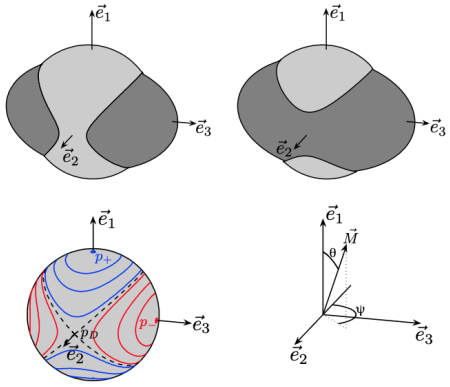
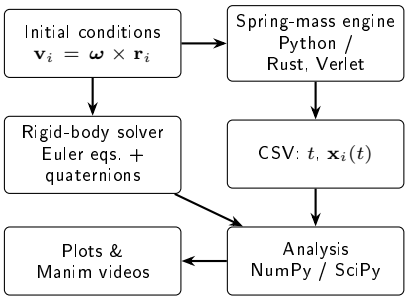



**Figure 2:** Top: the energy ellipsoid (dark) and the angular momentum sphere (light). Bottom left: polhodes on the sphere, with the separatrix dashed. Adapted from [9].

## 3. Method

### 3.1. Overview of the pipeline




### 3.2. The rigid body as point masses and springs

In our first approach no rigid body equations are coded at all. The body is built from a few point masses, and every pair of masses is joined by a stiff spring whose rest length is the initial distance between them (`rigid=True` in `System`). With $N$ masses there are $N(N-1)/2$ springs, so the six-mass octahedron has 15. Each spring pulls along the line joining its two masses, with a force proportional to its stretch plus an optional damping term proportional to the rate of stretching (listing 1). These forces are equal, opposite and central, so the total momentum and angular momentum are conserved, and damping only removes energy. Rigid body motion, Euler's equations and the flip are not put in by hand; they have to emerge from Newton's second law for the individual masses.

```
def _spring_force(self, x, v, t):
    F = np.zeros_like(x)
    for (i, j, k, c, rest) in self.springs:
        d = x[j] - x[i]  # points from i to j
        r = np.linalg.norm(d)
        s = k*(r - rest) + c*np.dot(v[j] - v[i], d)/r
        f = s*d/r
        F[i] += f
        F[j] -= f
    return F
```

**Listing 1:** Spring forces on all masses (`engine.py`, condensed).

The springs have to be stiff so that the body barely deforms. With $k = 10^5$ N/m and 1 kg masses they vibrate at $\omega_s \sim \sqrt{k/m} \approx 300$ rad/s, while the body itself turns at about 1 rad/s. The time step has to resolve these fast vibrations, which makes the model expensive. For this reason the engine, first written in Python, was ported with the same structure to Rust for the long runs with small time steps.

#### 3.2.1. Velocity Verlet

Newton's equations for the masses are integrated with the velocity Verlet scheme in its kick–drift–kick form,

$$\begin{aligned} \mathbf{v}_i^{n+1/2} &= \mathbf{v}_i^n + \tfrac{\Delta t}{2m_i}\mathbf{F}_i(\mathbf{x}^n) \\ \mathbf{x}_i^{n+1} &= \mathbf{x}_i^n + \Delta t\, \mathbf{v}_i^{n+1/2} \\ \mathbf{v}_i^{n+1} &= \mathbf{v}_i^{n+1/2} + \tfrac{\Delta t}{2m_i}\mathbf{F}_i(\mathbf{x}^{n+1}) \end{aligned} \tag{6}$$

Each mass gets half a velocity kick from the current forces, moves for a full step, and gets the second half kick from the forces at its new position. The forces from the end of one step are reused at the start of the next, so only one force evaluation is needed per step (listing 2). Verlet is second order, time-reversible and symplectic, so the energy error stays bounded over long runs instead of drifting [2, 11]. This is the very reason we chose to use this over RK4 which is more accurate over one step.

```
def step(self, dt):
    self.v += 0.5*dt*self.force_cached/self.m  # half kick
    self.x += dt*self.v                        # drift
    self.t += dt
    self.force_cached = self.total_force()
    self.v += 0.5*dt*self.force_cached/self.m  # half kick
```

**Listing 2:** One velocity Verlet step (`engine.py`; `RigidBody::step` in Rust is the same).

### 3.3. Direct rigid body solver with quaternions

While the spring model is general, but it is indirect and expensive: the body is only approximately rigid, and the stiff springs force a tiny time step. A truly rigid body needs much less information. Its state is just its angular velocity and its orientation, and nothing in it vibrates, so the time step only has to resolve the rotation itself. As an independent check we therefore also solve the rigid body equations directly (`quaternion.py`). The angular velocity $\boldsymbol{\Omega}$ in the body frame follows Euler's equations (eq. (2)). The remaining question is how to store the orientation.

*Choosing a representation.* Three Euler angles are the most familiar choice and use the minimum number of variables, but their equations of motion contain $1/\sin\theta$ and blow up when two rotation axes line up (This is something called gimble lock), and every step needs trigonometric functions. A $3 \times 3$ rotation matrix has to stores nine numbers for three degrees of freedom; numerical errors slowly make it non-orthogonal, so it stops being a pure rotation and has to be re-orthogonalised. We found that there is a very beautiful way of describing these rotations using Quaternions [7]. These are

complex numbers but in 4D. A rotation by an angle $\phi$ about a unit axis $\hat{\mathbf{n}}$ is written as

$$q = \left(\cos\tfrac{\phi}{2},\ \hat{\mathbf{n}}\sin\tfrac{\phi}{2}\right), \qquad |q| = 1, \tag{7}$$

a scalar and a vector part, and two quaternions are multiplied as

$$(a,\mathbf{a})\otimes(b,\mathbf{b}) = \big(ab - \mathbf{a}\cdot\mathbf{b},\ a\mathbf{b} + b\mathbf{a} + \mathbf{a}\times\mathbf{b}\big). \tag{8}$$

Combining two rotations is a single product, with 16 multiplications instead of the 27 of a matrix product.

*Equations of motion.* With this representation the orientation evolves as

$$\dot{q} = \tfrac{1}{2}\, q\otimes(0,\boldsymbol{\Omega}), \qquad \mathbf{r}_{\text{lab}} = q\otimes(0,\mathbf{r}_{\text{body}})\otimes q^{*}. \tag{9}$$

The first equation is linear in $q$ and contains only products and sums, with no trigonometric functions and no singular points, which makes it both cheap and robust to integrate. The only constraint is $|q| = 1$, which numerical errors break slowly and which is restored by simply dividing $q$ by its length. The second relation rotates any point of the body into the lab frame, which we use for plots and animations. In total the state is seven numbers, $\boldsymbol{\Omega}$ and $q$, and eqs. (2) and (9) are solved together (listing 3). The dimensions and values for the body used is a real tennis racket.

```python
def euler_equations(w):
    w1, w2, w3 = w
    return np.array([(I2 - I3)/I1*w2*w3,
                     (I3 - I1)/I2*w3*w1,
                     (I1 - I2)/I3*w1*w2])

def dq(q, w):
    return 0.5*quaternion_multiply(q, [0, *w])
```

**Listing 3:** Time derivatives of $\boldsymbol{\Omega}$ and $q$ (`quaternion.py`, condensed).

#### *3.3.1. Fourth-order Runge–Kutta*

This system has no stiff springs, so we integrate it with the fourth-order Runge–Kutta method (RK4),

$$\mathbf{y}_{n+1} = \mathbf{y}_n + \frac{\Delta t}{6}\left(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4\right), \tag{10}$$

with $\mathbf{y} = (\boldsymbol{\Omega}, q)$ and $\Delta t = 1\,\text{ms}$. $\boldsymbol{\Omega}$ and $q$ are advanced together, so each of the four stages evaluates both derivatives (listing 4). RK4 does not keep $|q| = 1$ exactly, so $q$ is renormalised after every step.

```python
def rk4(w, q, dt):
    k1, l1 = euler_equations(w), dq(q, w)
    w2 = w + 0.5*dt*k1
    k2, l2 = euler_equations(w2), dq(q + 0.5*dt*l1, w2)
    w3 = w + 0.5*dt*k2
    k3, l3 = euler_equations(w3), dq(q + 0.5*dt*l2, w3)
    w4 = w + dt*k3
    k4, l4 = euler_equations(w4), dq(q + dt*l3, w4)
    w = w + dt/6*(k1 + 2*k2 + 2*k3 + k4)
    q = q + dt/6*(l1 + 2*l2 + 2*l3 + l4)
    return w, q/np.linalg.norm(q)  # keep |q| = 1
```

**Listing 4:** One RK4 step for $\boldsymbol{\Omega}$ and $q$ (`quaternion.py`, condensed).

### *3.4. Observables from particle positions*

The spring engines only save the positions of the masses, 60 times per simulated second. Everything else is computed from these positions afterwards (`src/sim/`).

- **Body axes.** The masses sit in opposite pairs on the principal axes, so the unit vector from one mass of a pair to the other gives that body axis $\mathbf{e}_k$.
- **Velocities.** Computed with `np.gradient`, which takes second-order finite differences of the saved positions.
- **Angular velocity.** Each axis moves as $\dot{\mathbf{e}}_k = \boldsymbol{\omega}\times\mathbf{e}_k$; summing $\mathbf{e}_k\times\dot{\mathbf{e}}_k$ over the three axes gives $2\boldsymbol{\omega}$, so

$$\boldsymbol{\omega} = \frac{1}{2}\sum_{k=1}^{3}\mathbf{e}_k\times\dot{\mathbf{e}}_k, \qquad \Omega_k = \boldsymbol{\omega}\cdot\mathbf{e}_k. \tag{11}$$

- **Angular momentum.** $\mathbf{L} = \sum_i m_i\,\mathbf{r}_i\times\mathbf{v}_i$ about the centre of mass, projected onto the body axes to draw the polhode.
- **Flips.** A flip is counted each time the component of $\mathbf{L}$ along the middle axis changes sign.

For the quaternion solver $\boldsymbol{\Omega}$ and $q$ are the state itself, so nothing has to be reconstructed.

### *3.5. Implementation*

The spring engine was first written in Python (`engine.py`) and then rewritten in Rust with the same design. Both are small general-purpose engines: they know nothing about rigid bodies or Euler's equations, only about point masses and the forces between them.

*Python engine.* A `System` object stores the positions, velocities and masses of all particles as NumPy arrays. Masses are added with `add_mass`, springs with `add_spring`, and any other force (for example gravity or an external torque) with `add_force`, as a function that takes the current positions, velocities and time and returns the force on every mass. With `rigid=True` the system joins every pair of masses with a spring before the first step. At each step the forces from all these sources are summed and passed to the Verlet integrator (listing 2), so adding new physics never requires touching the integrator. The loop over springs runs in interpreted Python, which makes runs with $\Delta t = 10^{-5}$ s ($10^7$ steps) impractically slow.

*Rust engine.* The Rust version is a library split into small modules (listing 5). `Vector3` is a plain 3D vector with overloaded arithmetic, dot and cross products, so the physics code reads like the equations. A `RigidBody` owns its masses, springs and external forces, and its `step` is the same Verlet step as in Python. An `Engine` holds one or more bodies and the clock, and each experiment is a short program in `examples/` that builds a body and runs it (listing 6). Being compiled, the Rust engine is about 500 times faster than the Python one on the same run (table 1): a $10^7$-step run takes under a second instead of seven minutes.

```
src/
  math/vector3.rs     Vector3 and its operators
  body/mass.rs        Mass {m, x, v}
  body/rigid_body.rs  springs, forces, Verlet step
  physics/spring.rs   damped spring force
  physics/force.rs    external force closures
  engine.rs           bodies, clock, CSV export
examples/             one program per run
```

**Listing 5:** Structure of the Rust engine (`physics-engine-rs`).

```
let mut b = RigidBody::new(true, 1e5, 0.0); // rigid, k, c
for r in positions {
    b.add_mass(m, r, w.cross(&r)); // v = w x r
}
let mut u = Engine::new();
u.add_body(b);
u.export_csv("six_masses_rs.csv", 1e-5, 60.0, 100.0)?;
```

**Listing 6:** Setting up and running a simulation with the Rust engine (condensed from `examples/`).

*Output and analysis.* Both engines write the same CSV format: a short header with $\Delta t$, the output rate, the masses and the list of springs, followed by one row per frame with the time and all positions. The analysis and animation scripts in `src/sim/` therefore work with either engine. They compute the observables of section 3.4 with NumPy [3], and produce the plots and the Manim animations [8].

We used AI tools to write the code of creating the videos and export the csv file in this format after the runs. Every other piece of code in both the python and rust engine and the quaternions file is handwritten. We could not have made the beautiful videos demonstrating this effect without AI and consider this as fair use.

**Table 1:** Wall-clock time to integrate the octahedron for $T = 100\,\mathrm{s}$ with $k = 10^5\,\mathrm{N/m}$, same initial state. Both engines end at the same positions to six digits.

| $\Delta t$ (s) | Steps | Python (s) | Rust (s) |
|---|---|---|---|
| $10^{-3}$ | $10^5$ | 4.3 | 0.01 |
| $10^{-5}$ | $10^7$ | 429 | 0.73 |

This shows how terribly slow python is for computation whenever loops are involved.

## 4. Visualisation

Every run is turned into an animation with Manim [8] (`src/sim/draw*.py`). Each video shows the moving body next to live plots of the quantities from section 3.4, so the motion and the numbers can be followed together. The full videos are listed in Appendix A; this section shows stills from them.

Each script reads a CSV file written by one of the engines, rebuilds the body from the masses and springs listed in its header, and computes the body axes, $\boldsymbol{\Omega}$ and $\mathbf{L}$ frame by frame as in section 3.4. The playback speed is chosen so that the motion is easy to follow: the octahedron runs are sped up four times, the long damped run twelve times, and the 0.7 s racket toss is shown in slow motion. The videos are rendered at $3840 \times 2160$ pixels and 60 frames per second. Seeing the body and the plots side by side also served as a quick check of the simulations, for example that $\mathbf{L}$ really stays fixed in space while the body turns over.

Figures 3 and 4 follow the octahedron spun about its middle axis, the first as a still from the video and the second as a plot of $\mathbf{L}$ seen from the body, made from the same data. Figure 5 shows one toss of a tennis racket computed with the quaternion solver, and fig. 6 shows a damped body moving to its axis of largest moment.

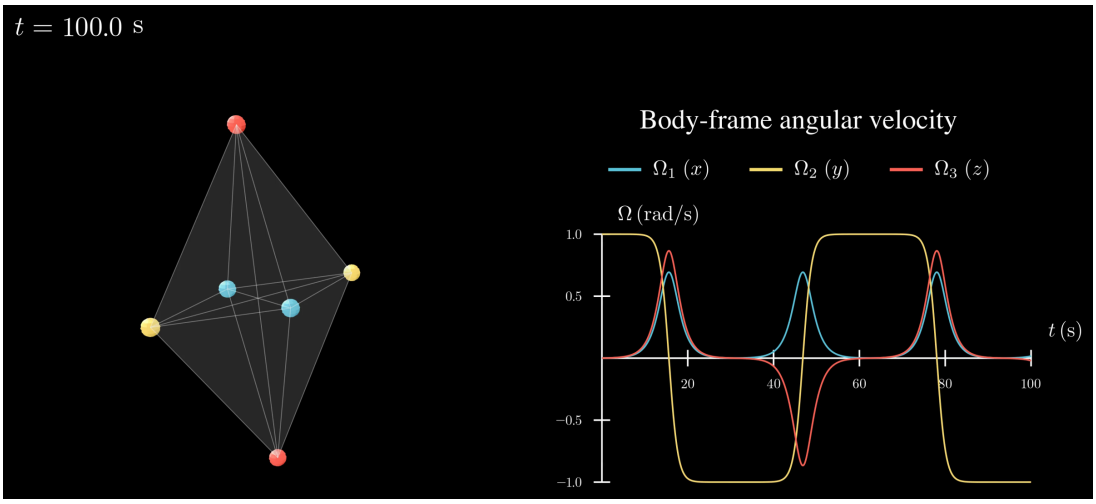



**Figure 3:** Octahedron (section 2.2) spun at 1 rad/s about its middle axis $y$ (spring engine, $t = 0$–100 s). Left: the body at the end of the run. Right: the body-frame angular velocity. $\Omega_2$ stays almost constant for long stretches and then suddenly reverses sign; each reversal is a flip. During a flip the spin briefly moves into the other two axes, giving the pulses in $\Omega_1$ and $\Omega_3$. The flips repeat regularly, here three times in 100 s.

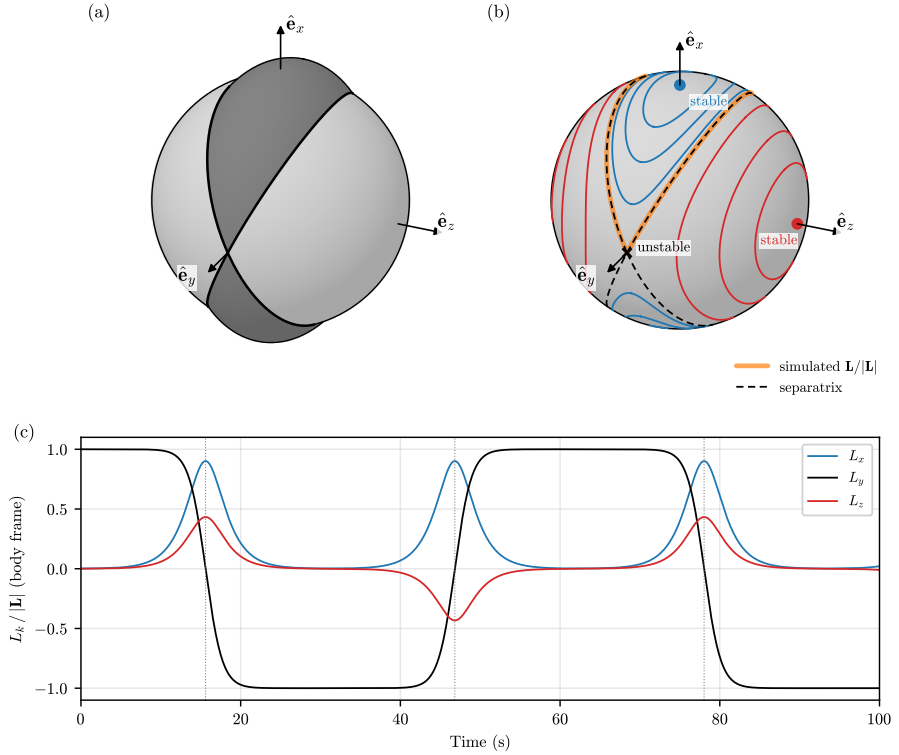



**Figure 4:** Angular momentum in the body frame for the same run, plotted from the simulation data. (a) The energy ellipsoid of this run (dark grey) cuts the sphere $|\mathbf{L}| = \text{const}$ (light grey); $\mathbf{L}$ must stay on the curves where they meet, which here are the separatrix crossing at the middle axis $\hat{\mathbf{e}}_y$. (b) Polhodes at other energies: blue curves circle the stable axis of largest moment $\hat{\mathbf{e}}_x$, red curves the stable axis of smallest moment $\hat{\mathbf{e}}_z$, and the dashed separatrix crosses at the unstable middle axis. The simulated $\mathbf{L}/|\mathbf{L}|$ over the whole 100 s (orange) lies on the separatrix; $2E/L^2$ changes by less than $10^{-5}$ of its value. (c) Body-frame components of $\mathbf{L}$: $L_y$ jumps between $\pm 1$ at each flip (dotted lines, every 31.2 s), while $\mathbf{L}$ passes through the $x$ side of the sphere.

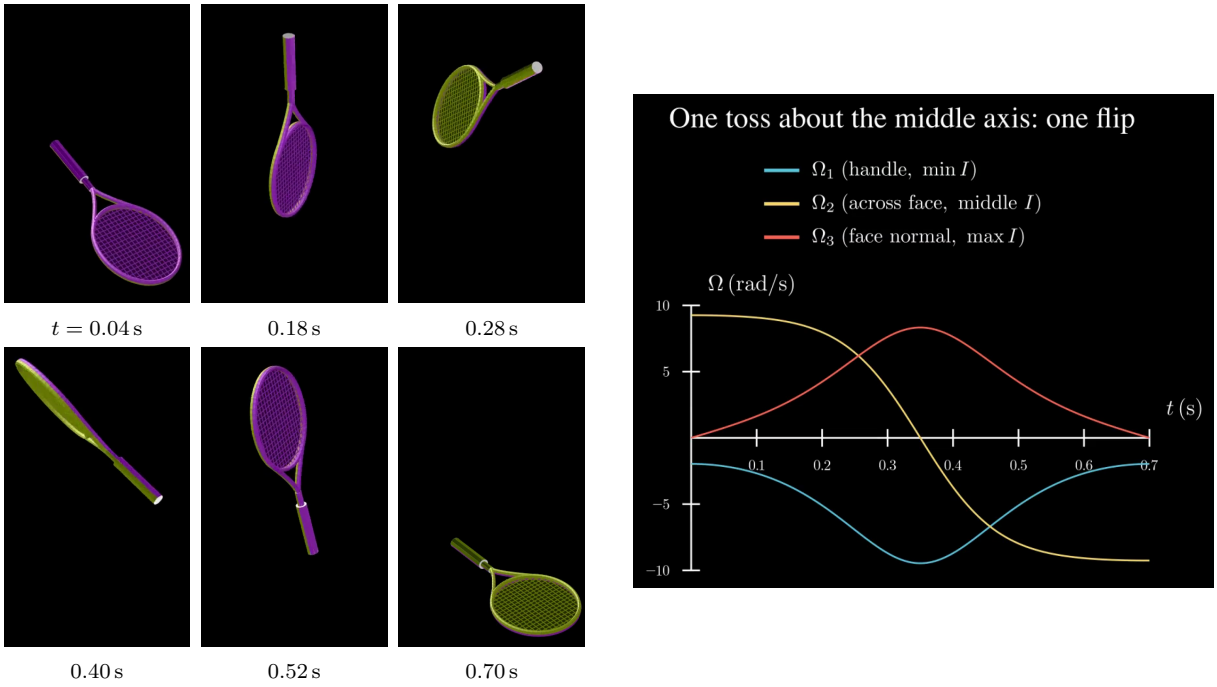



**Figure 5:** One toss of a tennis racket (`quaternion.py`, RK4 with quaternions). The racket is thrown spinning end over end, about its middle axis, with a small twist about the handle. The two sides of the strings are drawn in different colours (purple and yellow). Left: snapshots during the 0.7 s flight; the racket comes back with the other side of the face showing, having made a half turn about its handle. Right: the body-frame angular velocity. As $\Omega_2$ (middle axis) reverses, the spin passes through the handle ($\Omega_1$) and face-normal ($\Omega_3$) axes, the same pattern as one flip in fig. 3.

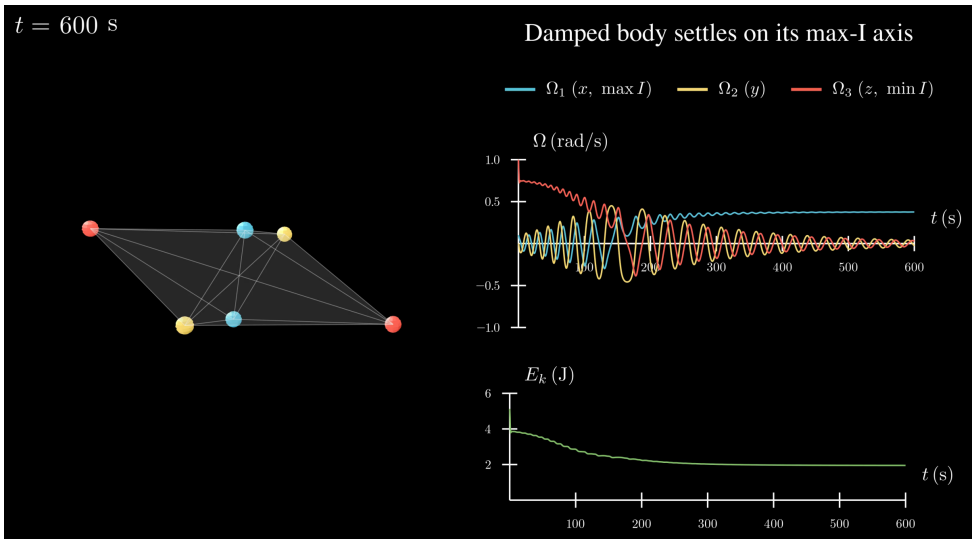



**Figure 6:** A damped body (soft springs, $k = 1\,\mathrm{N/m}$, $c = 0.7\,\mathrm{N\,s/m}$) started spinning about its axis of *smallest* moment $z$, slightly tilted toward $x$, shown at $t = 600\,\mathrm{s}$. Top right: the spin about $z$ ($\Omega_3$) dies away while the spin about $x$, the axis of largest moment, grows and settles. Bottom right: the kinetic energy falls and levels off near $L^2/2I_{\max} \approx 2.0\,\mathrm{J}$, the lowest energy possible at fixed $\mathbf{L}$ (section 6.4). Even rotation about the stable minimum axis is not stable once the body can lose energy.

## 5. Extension: energy loss in a non-rigid body

So far the body has been almost perfectly rigid, and its energy has been conserved. A real body is not perfectly rigid: it flexes slightly as it spins, and internal friction turns some of the energy of this wobbling into heat. As an extension of the project we study what this energy loss does to the rotation.

*The major axis rule.* No external torque acts, so $\mathbf{L}$ is still exactly conserved and only the energy $E$ decreases. For a fixed $L$, the energy of rotation about a principal axis is $E = L^2/2I$, which is smallest for the largest $I$. A body that keeps losing energy at fixed $L$ must therefore end up spinning about its axis of maximum moment of inertia, whatever axis it started on. This means that for a real, slightly flexible body only the major axis is truly stable: even spin about the minimum axis, which is stable for a rigid body (section 2.4), slowly drifts away. This is what happened to Explorer 1 in 1958, and it is why the Earth, which spins about its axis of largest moment, keeps a steady rotation.

*Model.* The spring model can show this directly, because its body is not perfectly rigid. We give every spring a damping coefficient $c$, the second term in listing 1, which resists the two masses moving apart or together. When the body turns rigidly the springs keep their length and the damping does nothing; only the wobbling is damped. Like the spring force, the damping force is central and equal and opposite, so it removes energy but conserves momentum and angular momentum exactly. This gives a useful check of the code: $|\mathbf{L}|$ must stay constant while $E$ falls.

We use soft springs, $k = 1\,\mathrm{N/m}$, and $c = 0.7\,\mathrm{N\,s/m}$, so that the body wobbles noticeably and settles within a few hundred seconds, instead of the far longer time a nearly rigid body would take. The soft springs also allow a larger time step, $\Delta t = 5\,\mathrm{ms}$. The octahedron starts spinning at $1\,\mathrm{rad/s}$ about $z$, its axis of smallest moment, tilted slightly towards $x$, $\boldsymbol{\omega}_0 = (0.1, 0, 1)\,\mathrm{rad/s}$, and is followed for 1000 s with both engines.

*Numerical details.* With a velocity-dependent force, velocity Verlet is no longer exactly symplectic, which is expected since the energy is meant to fall. The damping force computed at the end of a step uses the velocity available at that point, the half-step velocity $\mathbf{v}^{n+1/2}$ (listing 2). This adds a small error, of relative size about $c\Delta t/m \approx 0.004$ per step.

*Expected outcome.* The initial angular momentum is $\mathbf{L} = (I_x\omega_x, 0, I_z\omega_z) = (2.6, 0, 10)\,\mathrm{kg\,m^2/s}$, so $L \approx 10.3\,\mathrm{kg\,m^2/s}$ and $E_0 \approx 5.1\,\mathrm{J}$. If the body ends up spinning about $x$, it should turn at $\Omega_x = L/I_x \approx 0.40\,\mathrm{rad/s}$ with $E = L^2/2I_x \approx 2.05\,\mathrm{J}$. These values are only approximate, because the soft springs let the body stretch and change its moments of inertia while it spins. The run is shown in fig. 6 and analysed in section 6.4.

## 6. Results

All results are for the octahedron of section 2.2 simulated with the spring engines, unless stated otherwise. The figures and numbers in this section come from `plot_results.py` and `plot_flip_time.py` in `src/sim/`.

### 6.1. Stable rotation about the major and minor axes

To compare the three axes directly, we spun the body at $1\,\mathrm{rad/s}$ about each principal axis in turn, tilted by $\varepsilon = 0.01$ toward the next axis, and followed it for 100 s with the Python engine ($\Delta t = 1\,\mathrm{ms}$, $k = 10^5\,\mathrm{N/m}$). Figure 7 shows the angle between the spin axis of the body and the fixed angular momentum. About the axes of largest and smallest moment this angle only oscillates, between 0.2° and 0.44° about $x$ and between 1.49° and 1.65° about $z$, and does not grow over the whole run. About the middle axis the same small tilt, initially 0.29°, grows until the axis has turned over completely to 180°, and then returns, about every 48 s.

### 6.2. Rotation about the intermediate axis

The main run spins the body about its middle axis with the Rust engine ($\Delta t = 10^{-5}\,\mathrm{s}$, $k = 10^5\,\mathrm{N/m}$, 100 s); it is shown in figs. 3 and 4. The body flips three times, each flip being a sign change of the component of $\mathbf{L}$ along the middle axis. To test these times we solved Euler's equations (eq. (2)) for an ideal rigid body with SciPy's `solve_ivp` [12] at a relative tolerance of $10^{-12}$, starting from the same angular velocity as the simulation, $\Omega_k = L_k(0)/I_k$, which gives $\boldsymbol{\Omega}_0 \approx (0.0015, 1.0001, 0)\,\mathrm{rad/s}$. The two agree to within one output frame, $\frac{1}{60}\,\mathrm{s}$ (table 2), and the flips repeat every 31.2 s. Over the run the simulated $\mathbf{L}$ stays on the predicted polhode: $2E/L^2$ changes by less than $10^{-5}$ of its value, and $|\mathbf{L}|$ by $1.4 \times 10^{-4}$, the latter mostly from the finite-difference velocities.

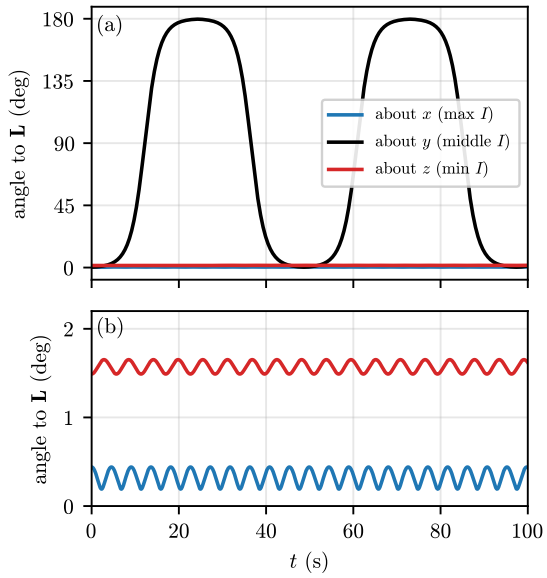



**Figure 7:** Angle between the spin axis of the body and $\mathbf{L}$ for spins about each principal axis with the same small tilt. (a) All three runs: only rotation about the middle axis $y$ turns over. (b) Zoom on the runs about $x$ and $z$, where the angle stays bounded.

**Table 2:** Flip times for spin about the middle axis.

| | 1st (s) | 2nd (s) | 3rd (s) |
|---|---|---|---|
| Spring engine | 15.60 | 46.82 | 78.03 |
| Rigid body (Euler) | 15.61 | 46.82 | 78.03 |

### *6.3. Flip time versus perturbation size*

Figure 8 shows the time to the first flip for 500 initial tilts $\varepsilon$ between $2 \times 10^{-7}$ and $10^{-4}$, computed with the Rust engine ($\Delta t = 1\,\mathrm{ms}$) for several spring stiffnesses, together with the rigid-body solution. The flip time grows without bound as $\varepsilon \to 0$, but only slowly: halving the tilt adds the same fixed amount of time. Fitting $t_{\mathrm{flip}} = \lambda^{-1} \ln(1/\varepsilon) + c$ to each set (table 3) confirms the logarithmic law of eq. (5); every fit has $R^2 > 0.9999996$. For the stiffest springs the fitted growth rate, $\lambda = 0.480383 \pm 0.000003\,\mathrm{s}^{-1}$, matches the prediction of eq. (4), $\lambda = 0.480384\,\mathrm{s}^{-1}$, within its uncertainty. Softer springs give a smaller $\lambda$ and later flips: the body stretches under its own rotation, which changes its moments of inertia.

### *6.4. Dissipation: relaxation to the major axis*

The model and the expected outcome are described in section 5; fig. 9 shows the damped run of the Rust engine. At the start $\mathbf{L}$ is 14.6° from the minimum axis $z$. Over the first few hundred seconds the body wobbles away from $z$ and settles on the

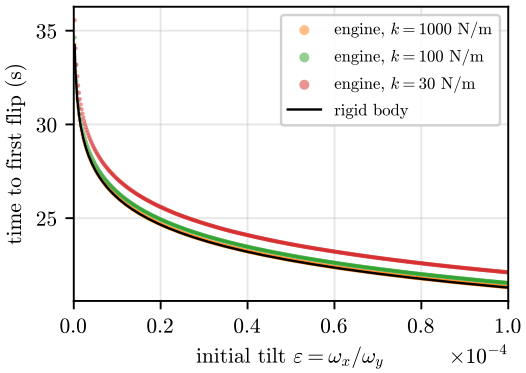



**Figure 8:** Time to the first flip against the initial tilt from the middle axis. Points: spring engine for three stiffnesses. Line: ideal rigid body from Euler's equations.

**Table 3:** Growth rate $\lambda$ fitted to the flip times, compared with eq. (4).

| $k$ (N/m) | $\lambda$ ($\mathrm{s}^{-1}$) | $\lambda/\lambda_{\mathrm{theory}}$ |
|---|---|---|
| $10^5$ | 0.48038 | 1.0000 |
| $10^3$ | 0.47981 | 0.9988 |
| 100 | 0.47466 | 0.9881 |
| 30 | 0.46191 | 0.9615 |
| Theory | 0.48038 | |

maximum axis $x$: by the end $\mathbf{L}$ is within 0.7° of $x$. Meanwhile $|\mathbf{L}|$ stays constant to $5 \times 10^{-5}$, as it should for central damping forces.

The kinetic energy first drops quickly from 5.13 J to about 3.9 J, because the springs start at their rest length: the spinning body first expands, storing part of the energy in the springs, and the fast oscillation this starts is damped away at once. After that it decays steadily and levels off at 1.948 J. This final value equals $L^2/2I_x = 1.948\,\mathrm{J}$ when $I_x$ is computed from the actual, stretched shape of the body at the end ($I_x = 27.4\,\mathrm{kg\,m^2}$ instead of 26 at rest), so the body ends in steady rotation about $x$ at $\Omega_x = 0.377\,\mathrm{rad/s}$. The estimates of section 5, 2.05 J and 0.40 rad/s, used the moment at rest, and the difference is exactly this stretching.

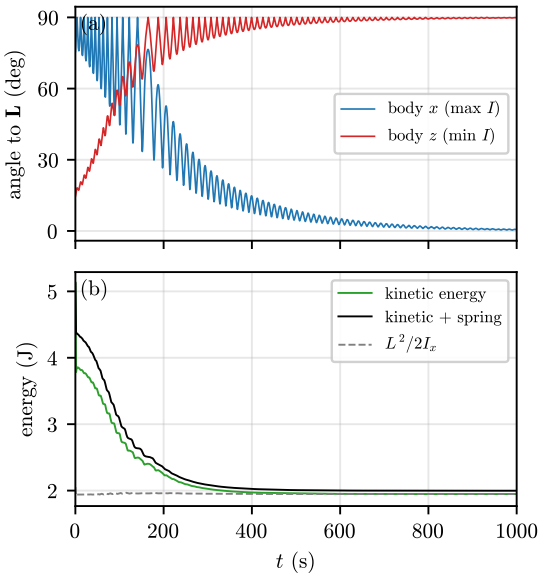



**Figure 9:** Damped body started near its minimum axis. (a) Angle between $\mathbf{L}$ and the body axes $x$ (max $I$) and $z$ (min $I$): the spin moves from $z$ to $x$. (b) Kinetic energy, kinetic plus spring energy, and the lowest energy possible at this $L$, $L^2/2I_x$, computed from the current shape of the body.

## 7. Validation and Error Analysis

The comparisons with the rigid body in section 6 already test the spring model. Here we check the solvers themselves. The numbers are computed by the script `plot_validation.py` in `src/sim/`.

### 7.1. Exact solution

Euler's equations have an exact solution in Jacobi elliptic functions [4]. The spin about the middle axis is proportional to sn, which changes sign every half period, so the body flips every $T/2$ with

$$T = 4K(k)\sqrt{\frac{I_1 I_2 I_3}{(I_3-I_2)(L^2-2EI_1)}}, \qquad k^2 = \frac{(I_2-I_1)(2EI_3-L^2)}{(I_3-I_2)(L^2-2EI_1)}, \tag{12}$$

in Landau's labelling $I_1 < I_2 < I_3$, and $K$ the complete elliptic integral of the first kind.

For the main run $1-k^2 = 4.9\times10^{-6}$; this close to the separatrix $1-k^2$ has to be computed directly, not as 1 minus $k^2$, or most of its digits are lost. The solution, evaluated with `scipy.special.ellipj` and `ellipkm1` [12], gives flips every $T/2 = 31.213\,\mathrm{s}$, the same times as in table 2. The simulated $\boldsymbol{\Omega}$ differs from it by a median of $2\times10^{-4}\,\mathrm{rad/s}$, mostly from the finite-difference velocities.

### 7.2. Conservation and convergence

Figure 10 follows the energy and angular momentum over 100 s. With velocity Verlet the energy error stays bounded, $2.5\times10^{-7}$ at $\Delta t = 1\,\mathrm{ms}$ and four times smaller at half the step, and $\mathbf{L}$ is conserved to round-off, $3\times10^{-14}$, because the spring forces are central. With RK4 the error is not bounded and builds up at each flip, but it is still only $8\times10^{-10}$ after 100 s at $\Delta t = 50\,\mathrm{ms}$.

Figure 11 confirms the order of both methods: $p = 4.00$ for RK4 and $p \to 2.00$ for Verlet. Verlet only reaches its order once $\Delta t$ resolves the spring vibrations, 160–775 rad/s. With the larger steps used in practice the phase of these vibrations is lost and the position error stays at a few µm, while the slow rotation that we measure stays accurate.

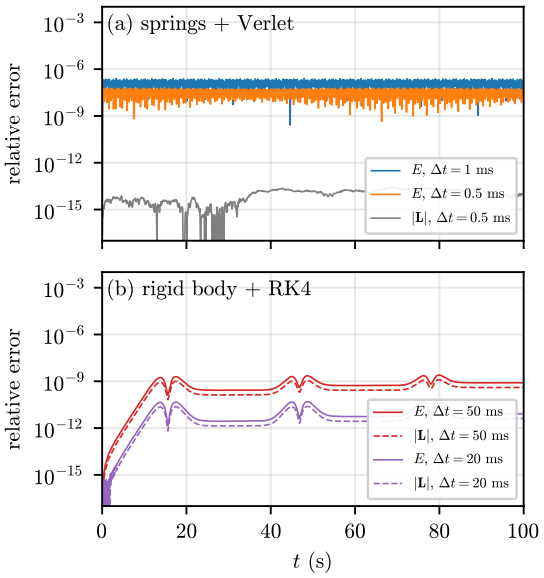



**Figure 10:** Relative change of the energy and angular momentum. (a) Spring engine with Verlet. (b) Rigid-body solver with RK4.

### 7.3. Error budget

The Python and Rust engines give bit-for-bit identical results after $10^5$ steps, and the stiffness error of the growth rate in table 3 falls as about $(1.2\,\mathrm{N/m})/k$. Table 4 collects the error sources for the main run. The largest is not in the simulation but in the analysis, which reconstructs velocities from positions saved at 60 Hz.

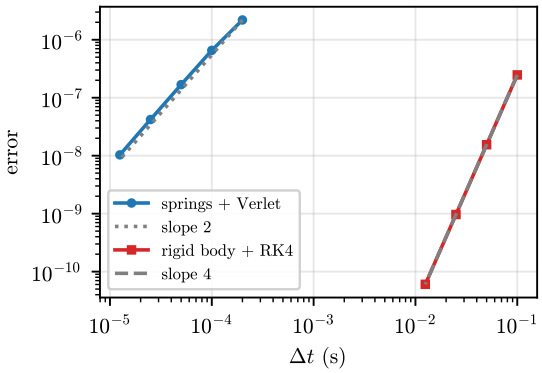



**Figure 11:** Error against time step: Verlet (positions after 1 s) and RK4 ($\boldsymbol{\Omega}$ and $q$ after 10 s), with slopes 2 and 4.

**Table 4:** Error sources for the main run ($\Delta t = 10^{-5}$ s, $k = 10^5$ N/m).

| Source | Relative size |
|---|---|
| Time step (energy error) | $\sim 10^{-11}$ |
| Finite stiffness ($\lambda$) | $\sim 10^{-5}$ |
| Velocities from 60 Hz output | $\sim 10^{-4}$ |
| Floating point | $\sim 10^{-14}$ |

## 8. Discussion

Both models reproduce the tennis racket effect. In the spring model nothing about rigid bodies is coded, yet the flips appear from Newton's law for six point masses alone, at the times predicted by Euler's equations and the exact solution, and with the growth rate of the linear theory. With damping added, the same model also shows the major axis rule.

The two models suit different jobs. The spring model is general: any shape can be built from masses, and effects beyond a rigid body, such as flexing and internal friction, come for free. Its cost is stiffness: the time step is set by spring vibrations several hundred times faster than the rotation, so the main run needed $10^7$ steps, and was only practical after moving from Python to Rust. The rigid-body solver with quaternions has only seven variables and no fast motion, so a step of 50 ms already keeps the energy to $10^{-9}$, but it can only describe a perfectly rigid body. The main limitation of our analysis was not in either solver but in recovering velocities from positions saved at 60 Hz, which sets the error floor of about $10^{-4}$.

### 8.1. Extensions

*Constraint solvers.* Instead of stiff springs, the distances between the masses can be held fixed as exact constraints. SHAKE [6] and its velocity form RATTLE [1] add Lagrange-multiplier forces to each Verlet step so that every bond length is restored exactly, while keeping Verlet's symplectic, time-reversible character [2]. This removes the fast vibrations, so the time step would be set by the rotation, as for the rigid-body solver, while keeping the generality of a particle model. The price is a set of nonlinear constraint equations to solve iteratively at every step.

*Chaos.* The free rigid body is integrable: with energy and $|\mathbf{L}|$ conserved its motion is regular, and even near the separatrix the flips are periodic, with a flip time that grows only logarithmically as the tilt shrinks. This changes once integrability is broken, for example by gravity acting on an asymmetric top, a periodic external torque, or a second body attached by a joint. The separatrix then turns into a thin chaotic layer, and the flips occur at irregular times. Such forces can be added to our engine without changing the integrator, and the chaos measured by the divergence of two runs started a tiny distance apart (a Lyapunov exponent).

*Lie groups.* Orientations form the rotation group $SO(3)$, a Lie group, and unit quaternions form its double cover. Our RK4 step moves $q$ off the unit sphere and projects it back by normalising, so it neither stays on the group nor preserves the structure of the equations, which is why its energy error accumulates (fig. 10). Lie-group integrators instead update the orientation with the exponential map, $q_{n+1} = q_n \otimes \exp\left(\frac{1}{2}\boldsymbol{\Omega}\Delta t\right)$, which stays on the group exactly. Splitting the free rigid body into rotations about its three axes, each of which can be solved exactly, gives symplectic methods that conserve $|\mathbf{L}|$ exactly and keep the energy error bounded [2], the same advantage Verlet gives the spring model.

## Use of AI tools

AI tools were used in the initial drafts of this report. We also used AI to make all the beautiful videos using manim as already mentioned above. We used it make some of the plots shown in results and validation. All the code pertaining to the actual physics engine and everything before calling the function to generate the csv file is done by us. We consider this as fair use.

## References

[1] H. C. Andersen, «Rattle: a "velocity" version of the shake algorithm for molecular dynamics calculations», Journal of Computational Physics **52**, 24–34 (1983).

[2]E. Hairer, C. Lubich, and G. Wanner, *Geometric numerical integration: structure-preserving algorithms for ordinary differential equations*, 2nd (Springer, 2006).

[3]C. R. Harris et al., «Array programming with NumPy», Nature **585**, 357–362 (2020).

[4]L. D. Landau and E. M. Lifshitz, *Mechanics*, 3rd, Vol. 1, Course of Theoretical Physics (Butterworth-Heinemann, 1976).

[5]L. Poinsot, *Théorie nouvelle de la rotation des corps* (Bachelier, Paris, 1834).

[6]J.-P. Ryckaert, G. Ciccotti, and H. J. C. Berendsen, «Numerical integration of the cartesian equations of motion of a system with constraints: molecular dynamics of n-alkanes», Journal of Computational Physics **23**, 327–341 (1977).

[7]G. Sanderson and B. Eater, *Quaternions and 3d rotation, explained interactively*, YouTube (3Blue1Brown), 2018.

[8]The Manim Community Developers, *Manim – mathematical animation framework*, 2024.

[9]L. Van Damme, P. Mardešić, and D. Sugny, «The tennis racket effect in a three-dimensional rigid body», Physica D: Nonlinear Phenomena **338**, arXiv:1606.08237, 17–25 (2017).

[10]Veritasium, *The bizarre behavior of rotating bodies*, YouTube, 2019.

[11]L. Verlet, «Computer "experiments" on classical fluids. I. Thermodynamical properties of Lennard-Jones molecules», Physical Review **159**, 98–103 (1967).

[12]P. Virtanen et al., «SciPy 1.0: fundamental algorithms for scientific computing in Python», Nature Methods **17**, 261–272 (2020).

[13]Wikipedia contributors, *Tennis racket theorem*, Wikipedia, The Free Encyclopedia, (2026) https://en.wikipedia.org/wiki/Tennis_racket_theorem (visited on 11/13/2026).

## Appendix A. Supplementary videos

The videos are in the `videos/` folder of the project repository.

**Table A.5:** Supplementary Manim videos.

| Video | Content | File |
|---|---|---|
| V1 | Octahedron spun about its middle axis, with live $\Omega_k(t)$ | link |
| V2 | Same run: the body with the fixed $\mathbf{L}$, and $\mathbf{L}$ on the sphere and energy ellipsoid | link |
| V3 | Damped body settling on its major axis | link |
| V4 | Tennis racket toss, quaternion solver | link |